

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 08 · LECTURE NOTES

Trees and Connectivity

The structure every data structure course assumes.

MODULE OUTCOMES

Give the equivalent characterisations of a tree.

Prove the edge-count theorem.

Explain spanning trees and why they exist.

Explain rooted trees, height, and node counts.

Connect this to the data structures ahead.

1 What a tree is

A TREE IS A CONNECTED GRAPH WITH NO CYCLES — AND FOUR OTHER STATEMENTS THAT ARE EQUIVALENT TO IT

Connected and acyclic is the usual definition. **Connected with exactly $n - 1$ edges** is equivalent to it. **Acyclic with exactly $n - 1$ edges** is equivalent to it as well — note that each of those drops a condition and adds the edge count.

And: there is a unique path between every pair of vertices — which is the characterisation that explains why trees are useful for routing and addressing, since there is never a choice to make.

And: minimally connected — removing any edge disconnects the graph, and symmetrically **adding any edge creates exactly one cycle**, which is the characterisation §3 uses.

All five describe the same property, and proving the equivalences is the standard exercise — which is worth doing once, because **later courses switch between them without comment** and expect you to follow.

2 The edge count

CLAIM: a tree with $n \geq 1$ vertices has $n - 1$ edges.

BASE CASE. $n = 1$: one vertex, no edges. $1 - 1 = 0$.

INDUCTIVE STEP. Suppose every tree on n vertices has $n - 1$ edges. Let T be a tree on $n + 1$ vertices.

- 1 T has a leaf (a vertex of degree 1).
 WHY: follow a longest path in T ; its endpoint cannot continue (that would make a longer path) and cannot have another edge back into the path (that would make a cycle), so it has degree 1.
 <-- this is the step people skip, and it is the only interesting part of the whole proof
- 2 Delete that leaf and its single edge. The result is still connected (the leaf lay on no path between other vertices) and still acyclic (deleting never creates a cycle). So it is a tree on n vertices.
- 3 By the inductive hypothesis it has $n - 1$ edges. Add back the edge we removed: n edges.

So T has $(n + 1) - 1$ edges. By induction, done. []

Step 1 is the only interesting part of this proof, and it is the step most people assert rather than prove — 'obviously a tree has a leaf' — **which is Module 03's five errors and Module 01 §1's 'obviously' warning, both in miniature. Write this proof out yourself;** it is the model for essentially every tree argument in the rest of the program, and the delete-a-leaf-and-apply-the-hypothesis move recurs constantly.

3 Spanning trees

EVERY CONNECTED GRAPH CONTAINS A SPANNING TREE, AND THE PROOF IS AN ALGORITHM

A spanning tree is a subgraph that is a tree and includes every vertex of the original — the minimal structure that keeps the graph connected, with no redundancy left in it.

And one always exists, which is proved by construction: while any cycle remains, delete one edge of it — connectivity is preserved because the rest of that cycle still joins the deleted edge's endpoints — and stop when the graph is acyclic, which must happen since each step removes an edge.

Which is a constructive existence proof (Module 02 §4's first flavour) — and the construction is the algorithm, which is a pattern worth noticing and which CSCE 629 relies on repeatedly: a good existence proof usually hands you an implementation.

Plus an immediate corollary: a connected graph on n vertices has at least $n - 1$ edges, since it contains a spanning tree — and that bound is used constantly in complexity arguments to justify writing $O(E)$ rather than $O(V + E)$.

4 Rooted trees

- Pick any vertex as the root, and every other vertex has a unique parent — because there is a unique path to the root (§1's fourth characterisation), and the parent is the first step along it. An unrooted tree becomes a data structure the moment you choose a root.
- A binary tree of height h has at most $2^{h+1} - 1$ nodes (a geometric sum over the levels), so a tree with n nodes has height at least $\log_2(n+1) - 1$ — which is the lower bound every balanced-tree argument rests on, and it is a counting fact rather than an algorithmic one.
- And at least $h + 1$ nodes, achieved by a path — which is precisely why unbalanced trees degrade to linear time: a tree can be as tall as it has nodes.
- A binary tree with n internal nodes has $n + 1$ external (leaf) nodes, which is a double count (Module 06 §3) and is used in parsing arguments and in the analysis of prefix codes.
- And the sum of the node depths determines the total search cost, which is what CSCE 629's balanced trees are actually optimising — not the height alone, though bounding the height bounds the sum.

AND THIS IS WHERE THE PROGRAM'S DATA STRUCTURES START

Binary search trees, heaps, tries, B-trees, and bounding volume hierarchies are all rooted trees — and every performance bound on them is one of §4's counting facts dressed in the vocabulary of the particular structure.

Including the graphics ones: a bounding volume hierarchy is a binary tree, and its traversal cost is its depth (CSCE 647 Module 04's acceleration structures, and CSCE 620's spatial partitioning) — which is why a badly built BVH costs you the same way a badly built search tree does.

So the whole balanced-tree machinery of CSCE 629 is about forcing the height down toward the lower bound — which this module establishes and that course achieves, and seeing the division of labour makes both easier.

Which is the pattern of this entire prerequisite: the bound comes from counting, and the algorithm is the attempt to meet it — Module 09 makes that division explicit and it holds for the rest of the program.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapter 12 (free)	§§1 to 3, free — trees and the equivalence proofs, done carefully.
MIT 6.042J, the trees lecture (free video)	§§1 and 2 — with the leaf argument of §2 made explicit.
CLRS, the tree and heap chapters (library copy)	§4 — where these counting facts become data structure bounds, which is CSCE 629.
Sedgewick & Wayne, Algorithms (free site and lectures)	§4 implemented — free, with code, and a good way to see the bounds hold empirically.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Prove two of the five characterisations equivalent**, both directions.
- 2 **Prove every tree has a leaf**, carefully.
- 3 **Write the edge-count induction out in full**, with all three of Module 03's requirements visible.
- 4 **Construct a spanning tree** by the deletion algorithm.
- 5 **Count the spanning trees** of a small graph by hand.

- 6 **Derive the maximum node count** of a binary tree of height h .
- 7 **Derive the minimum height** for n nodes.
- 8 **Prove the internal-external node identity** by double counting.
- 9 **Build an unbalanced BST** and measure its search cost against the bound.
- 10 **Find the tree bound** behind one data structure you already use.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five equivalent definitions of a tree.
- 2 Why does every tree have a leaf?
- 3 Prove the edge-count theorem.
- 4 Which step is the interesting one, and why is it skipped?
- 5 What is a spanning tree, and why does one always exist?
- 6 What is the corollary about edge counts?
- 7 Give the node-count bounds for a binary tree of height h .
- 8 What is the minimum height for n nodes, and where is it used?
- 9 State the internal-external identity.
- 10 State the module's closing pattern.